SEMINAR PROJECT REPORT

Learning Multi-Hop Table Reasoning through RL: PPO-Based Entity Selection for Decision Making

Problem Statement, Literature Review, MDP Formulation,
Evaluation Suite and RL Architecture

Arijit Paul • Advay Bapat
IIT Bombay • Status: Week 4 of 10

Executive summary

We study multi-table, multi-hop question answering (MMQA, ICLR 2025), where answering a question requires following primary/foreign-key links across tables that are far too long for an LLM to read. We cast evidence selection as a short-horizon, deterministic MDP: a small learned policy chooses *anchors* (column–value predicates), *joins* (schema edges) and when to *stop*; the SQLite engine executes each decision; and a frozen LLM reader answers from the resulting sub-table under a hard token budget. Executing prefixes of the gold SQL gives exact, annotation-free intermediate supervision, which we use as potential-based (policy-invariant) reward shaping. The central empirical question is *when a learned critic pays off*: we compare PPO (learned per-state critic) against a ladder of alternatives that differ only in the granularity and accuracy of the baseline—REINFORCE (batch mean), GRPO (per question), VinePPO-style Monte-Carlo values (per state, unbiased) and PPO-KL.

Contents

A Problem Statement **2**
A.1 The problem in one paragraph . . . 2
A.2 Data model . . . 2
A.3 What a question is, formally . . . 2
A.4 Why this is hard . . . 3
A.5 The selection–reading decomposition . . . 3
A.6 What "entity selection" means here . . . 4
A.7 Question types and what sufficiency requires . . . 4
A.8 Why a sequential, learned policy, and why RL . . . 4
A.9 Supervision and availability . . . 5
A.10 Scope and non-goals . . . 5

B Literature Review **5**
B.1 MMQA — the task, the data, the oracle . . . 5
B.2 Chain-of-Table — the prompted sequential planner . . . 6
B.3 SCE — the entity-selection MDP template . . . 6
B.4 StepSearch — dense step-wise rewards for PPO . . . 7
B.5 GRPO on tables — why critic-free RL stalls . . . 7
B.6 Rewarding Progress — the theory behind dense rewards . . . 7
B.7 VinePPO — is the critic accurate? . . . 8
B.8 JAR — inferring join edges . . . 8
B.9 TableRAG — grounding anchors in large tables . . . 9
B.10 TabSQLify — the one-shot select-then-read competitor . . . 9
B.11 Synthesis and research gap . . . 9

C MDP Formulation (Final) **10**
C.1 Design decisions . . . 10
C.2 Episode instance . . . 10
C.3 Formal definition . . . 11
C.4 Reward . . . 12
C.5 Oracle construction and data integrity . . . 12
C.6 Worked episode . . . 13
C.7 Properties of the MDP . . . 13

D Evaluation Metric Suite **13**

E System and RL Architecture **14**
E.1 Overall pipeline . . . 15
E.2 Policy network . . . 15
E.3 The PPO actor–critic skeleton . . . 16
E.4 PPO variants: a ladder of baselines . . . 17
E.5 Baselines and experimental protocol . . . 18
E.6 Compute budget . . . 19

References **19**

A Problem Statement

A.1 The problem in one paragraph

We are given a natural-language question Q and a small relational database D—two or three tables, each with hundreds to thousands of rows, linked by primary-key/foreign-key relationships. Answering requires combining evidence across tables by following those links. A frozen LLM (the *reader*) will produce the answer, but it cannot usefully read the full tables: they are too long, and its accuracy collapses well before its context window fills. Our problem is to learn a *selection policy* that decides, step by step, which tables, keys and rows to extract, so that the reader receives a compact sub-database that is **sufficient** (the answer is still derivable from it), **small** (it fits a token budget), and **faithful** (it was obtained through the correct relational path, not by coincidence).

A.2 Data model

- **Relational database.** $D = \{T_1, \ldots, T_m\}$ with $m \in \{2, 3\}$ in MMQA. MMQA tables average about 6 columns and ∼1,833 rows (2-table subset) and 4.8 columns and ∼1,369 rows (3-table subset).
- **Keys.** A *primary key* (PK) uniquely identifies each row of its table; a *foreign key* (FK) in T_j references a PK in T_i. An FK→PK pair defines a legal *join edge* $e = (T_j.c_{\text{fk}} \to T_i.c_{\text{pk}})$.
- **Schema graph.** $G = (V, E)$ with tables as nodes and join edges as edges. Every multi-table question corresponds to a path or tree in G.
- **Cardinality.** Traversing PK→FK is 1-to-many (one department, many management records); the reverse is many-to-1. Intermediate evidence can grow or shrink with each hop, and the traversal direction determines how large it gets.
- **Junction (bridge) tables.** Spider-derived databases often model many-to-many relations through a table holding little besides two FKs, e.g. `Management(department_ID, head_ID)`. This matters a great deal (§A.3).

A.3 What a question is, formally

Every MMQA question comes with a gold SQL query $q^\star$ of (almost always) the form

$$q^\star(D) = \gamma\Big(\pi_A\big(\sigma_P(T_{i_1} \bowtie_{e_1} T_{i_2} \bowtie_{e_2} \cdots \bowtie_{e_k} T_{i_{k+1}})\big)\Big), \tag{1}$$

where the joins $\bowtie_{e_j}$ are the *hops*, σ_P applies the *anchor* predicates (e.g. `born_state = 'Alabama'`), π_A projects the answer columns, and γ is an optional final operation (DISTINCT, COUNT, SUM/AVG, ORDER BY … LIMIT, NOT IN/EXCEPT). The gold answer is $a^\star = q^\star(D)$. The number of joins is $k = 1$ for the 2-table subset and $k = 2$ for the 3-table subset. MMQA labels questions as Select (1,289 / 346 in the 2-/3-table subsets), Numerical (889 / 289), List (214 / 44) and Count (200 / 42); these map onto different forms of γ, which matters (§A.7).

Worked example

"What are the distinct creation years of the departments managed by a secretary born in state 'Alabama'?"

```
SELECT DISTINCT T1.creation FROM department AS T1
JOIN management AS T2 ON T1.department_ID = T2.department_ID
JOIN head AS T3 ON T2.head_ID = T3.head_ID
WHERE T3.born_state = 'Alabama'
```

As a sequence of decisions:

(1) anchor on `Head.born_state = 'Alabama'` $\Rightarrow$ head_ID $= 1$;

(2) join `Head.head_ID` $\rightarrow$ `Management.head_ID` $\Rightarrow$ department_ID $= 7$;

(3) join `Management.department_ID` $\rightarrow$ `Department.department_ID` $\Rightarrow$ one row;

(4) project `creation`, stop; the reader answers "1903".

Three general lessons follow:

1. **The bridge hop is latent.** The question never mentions `Management`; the only cue is the verb "managed". The path passes through a table with no lexical footprint in the question—a structural reason LLMs fail at join selection.
2. **Order is a choice.** Joins are commutative and associative, so starting from `Department` gives the same answer but carries every department row forward. Starting from the selective anchor keeps each intermediate set tiny. This is the *join-ordering* problem that query optimisers solve; our policy effectively learns a selectivity-aware plan under a token budget.
3. **The needed evidence is tiny**—one row per table out of thousands. That ratio is the whole opportunity.

A.4 Why this is hard

1. **Length.** Roughly 1,800 rows $\times$ 6 cols$\times$ $\sim$ 3 tokens/cell $\approx$ 30k tokens per table, i.e. 60–90k tokens per question. MMQA reports that multi-table QA accuracy degrades gently up to about 800 rows and then drops sharply. Selection is mandatory, not cosmetic.
2. **Key identification.** Even o1-preview selects foreign keys correctly only 34.17% (2-table) and 40.78% (3-table) of the time, versus 95.3% and 97.5% for humans.
3. **Reasoning order**—MMQA lists it explicitly as a core challenge, and it also determines evidence size.
4. **Latent hops** through bridge tables.
5. **Question–schema lexical mismatch.** In MMQA's paraphrase study, removing column-related wording lowered every model's accuracy; anchor selection needs semantic matching, not string lookup.
6. **Spurious paths.** A wrong join path can still yield the right answer by coincidence (especially for Count questions), so answer accuracy alone cannot separate correct reasoning from lucky reasoning.

A.5 The selection–reading decomposition

The system has two components: a small learned **selector** π_θ that produces a trajectory $\tau = (a_1, \ldots, a_T)$ whose execution on D yields a sub-database $S_\tau \subseteq D$, and a frozen **reader** $\mathcal{R}$ (greedy decoding) mapping $(Q, S_\tau) \mapsto \hat{a}$. The objective is

$$\max_\theta \; \mathbb{E}_{(Q,D)\sim\mathcal{D}} \, \mathbb{E}_{\tau\sim\pi_\theta(\cdot|Q,D)} \Big[\mathbb{1}\big[\mathcal{R}(Q, S_\tau) \equiv a^\star\big] \Big] \quad \text{s.t.} \quad \text{tok}(S_\tau) \le B, \;\; |\tau| \le H, \tag{2}$$

with $\equiv$ normalised answer equivalence, B a token budget well below the $\sim$800-row elbow, and H a small horizon. Freezing the reader makes every change in accuracy attributable to selection.

Sufficiency. $S \subseteq D$ is *sufficient* for (Q, D) if $q^\star(S) = q^\star(D)$. Sufficiency depends only on the selection, not the reader, and it separates the two failure modes:

$$P(\text{correct}) = \underbrace{P(S \text{ suff.})}_{\text{selector}} \cdot \underbrace{P(\mathcal{R} \text{ correct} \mid S \text{ suff.})}_{\text{reader}} + \underbrace{P(S \text{ insuff.})\, P(\mathcal{R} \text{ correct} \mid S \text{ insuff.})}_{\text{luck / priors}}. \quad (3)$$

This is why we measure sufficiency directly (sub-table answer coverage, Part D); the reader factor on gold sub-tables is the *oracle upper bound*.

Database-theory view. For monotone select–project–join queries the minimal sufficient sub-database is the *lineage* of the answer—exactly the rows contributing to some answer tuple. The selector is trying to recover the lineage of the answer without knowing the query.

A.6 What "entity selection" means here

Level	Object selected	Knowledge-graph analogue	Example
Table	which table to move to	next entity type	Head → Management
Key / column	which join edge (FK→PK pair)	relation r	`Head.head_ID→Management.head_ID`
Row / value	which rows' keys are carried forward	entity e	head_ID ∈ {1}

The *entity set* E_t is the set of rows currently consistent with all decisions. A join maps $E_t \mapsto E_{t+1}$ deterministically by executing it in SQLite: the policy never guesses rows directly; it chooses anchors and edges and execution propagates the consequences. This determinism is what makes execution-based rewards exact.

A.7 Question types and what sufficiency requires

Whether a small selection can be sufficient depends on whether the final operation γ is *monotone* (adding rows can only add answers).

Type	Monotone	Sufficiency requires	Implication
Select / List	yes	contain the lineage; extra rows harmless	recall is what counts
Count / Numerical	no	the *complete* qualifying set, exactly	recall-critical, exact
Superlative / top-k	no	the entire comparison set	needs full (projected) column
Negation (NOT IN)	no	the full key column of the excluded table	must "prove absence"

Consequences: one budget does not fit all (pinpoint questions need a few full rows, comparison questions a narrow projection over many rows), and aggregation over hundreds of rows is a known weakness of 7B readers. Both are resolved in Part C.

A.8 Why a sequential, learned policy, and why RL

Sequential beats one-shot. A one-shot selector commits before seeing any intermediate result; a sequential policy can condition on what it observes (an anchor matching 900 rows calls for a second anchor before joining; one matching a single row allows joining freely).

Any fixed plan is a policy that ignores observations, so the best adaptive policy is at least as good as the best non-adaptive one, strictly better whenever the best next decision depends on intermediate cardinalities.

Learned beats prompted. Prompted planners (Chain-of-Table) assign no credit across steps; frontier prompted models get the key decisions right less than half the time (MMQA).

RL beyond imitation. Behaviour cloning (BC) on gold trajectories is our warm start and a baseline, but (i) many trajectories are correct (join commutativity) and imitation penalises valid alternatives; (ii) SQL execution and the frozen reader make the objective non-differentiable; (iii) imitation ignores the budget and the particular reader; (iv) cloned policies compound errors on states never seen in training.

A.9 Supervision and availability

Signal	Source	Training	Test
Question Q, tables D	MMQA	✓	✓
Gold answer $a^\star$	human-annotated	reward	evaluation only
Gold SQL $q^\star$	template-generated	oracle + dense reward	evaluation only
Gold intermediate sets $E_t^\star$	executing prefixes of $q^\star$	✓	evaluation only
PK/FK annotations	human experts	reward only	**not used** (edges inferred)

Because $q^\star$ is executable, the exact correct evidence at every hop is free—a structural advantage over text multi-hop QA, where it must be approximated.

A.10 Scope and non-goals

In scope: learning anchor/join/stop decisions over a multi-table database under a token budget; comparing RL algorithms for that policy; measuring sufficiency, faithfulness and cost alongside accuracy. **Out of scope:** training the reader; open-domain retrieval over all databases (a distractor-table setting is the most we add); numeric computation by the policy. **Positioning against text-to-SQL:** SQL generation is all-or-nothing and MMQA shows generated multi-table SQL is often wrong; paraphrased predicates are better handled by a reader looking at candidate rows; and stepwise selection degrades gracefully. A text-to-SQL extraction baseline is included (Part E).

B Literature Review

We organise the literature by the role each work plays in our pipeline: the task and its failure mode (B.1), sequential formulations (B.2–B.3), RL machinery and reward design (B.4–B.7), training-free tools for the action space (B.8–B.9), and the closest competing pipeline (B.10). Works surveyed but not pursued this semester (e.g. whole-LLM RLVR on tables such as Table-R1, and Text2SQL-GRPO transfer) are omitted.

B.1 MMQA — the task, the data, the oracle

Wu, Yang, Li, Ji, Okumura, Zhang. "MMQA: Evaluating LLMs with Multi-Table Multi-Hop Complex Questions." ICLR 2025 (Oral).

Construction. 5,000 samples drawn from Spider databases; multi-table SQL synthesised from 45 hand-written templates; GPT-4-turbo turns SQL into questions; two experts annotate PKs/FKs and answers (a third resolves disagreements). Each of the 3,312 instances (2,591 two-table, 721 three-table) carries a question, gold SQL, gold answer and key annotations.

Findings. Best model (o1-preview, one-shot): 50.78 / 48.28 EM on 2-/3-table, vs. human 89.8 / 92.3; FK selection 34.17 / 40.78% vs. human 95.3 / 97.5%. An accuracy elbow appears beyond ∼800 rows.

MTR (their retriever). GPT-4-turbo decomposes the question into sub-questions; retrieval runs as rounds of single-table retrieval with a fine-tuned TableLlama-7B/SGPT-5.8B scoring question–table relevance α, multiplied by a binary column-overlap table–table score β; iteration stops when overlap is zero. *Partial Match* is a binary GPT-4-turbo judgement whose Pearson correlation with human checks falls with weaker answering models (0.885 for o1-preview, 0.827 GPT-4, 0.778 GPT-3.5).

What we take

Gold SQL is our reward oracle (parse with `sqlglot`, execute prefixes in SQLite). The ∼800-row elbow justifies a hard token budget. MTR is effectively a greedy, hand-scored sequential policy—a structural reference point (we reproduce a cheaper JAR-style structural baseline instead, B.8). PM is reported only as a secondary, audited metric.

B.2 Chain-of-Table — the prompted sequential planner

Wang et al. "Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding." ICLR 2024.

An LLM iteratively selects an atomic operation (add-column, select-row, select-column, group-by, sort-by, or end) with `DynamicPlan`, conditioned on the current transformed table and the operation chain, then fills arguments with `GenerateArgs`; a final query answers. Single-table (WikiTQ, TabFact, FeTaQA); nothing is learned.

What we take

(i) Factorised actions $\pi(a \mid s) = \pi(\text{type} \mid s)\,\pi(\text{arg} \mid \text{type}, s)$; (ii) state = intermediate table + operation history; (iii) a *prompted-planner baseline* that uses the same frozen reader to act over our action API—the cleanest "learned vs. prompted" comparison.

B.3 SCE — the entity-selection MDP template

Huang, Liu, Liu, Hou, Dai. "SCE: Semantic Consistency Enhanced Reinforcement Learning for Multi-Hop Knowledge Graph Reasoning." Findings of EMNLP 2025.

MDP. State $s_t = (e_t, r_q, h_t)$ (current entity, query relation, GRU-encoded history); actions are outgoing (relation, entity) edges plus a self-loop acting as *stop*; policy $\pi_\theta(a_{t+1} \mid s_t) = \text{softmax}(A_t\, W_2\, \text{ReLU}(W_1 s_t))$; trained with REINFORCE. *Problem addressed:* spurious paths that reach correct answers for wrong reasons. *Reward:* accuracy $R_a = R_b + (1 - R_b) f_{\text{ConvE}}$, times a semantic-consistency score combining local cosine similarity and global "moves-closer" fractions in TransE space; final $R = R_a + R_a \cdot SC$. Runtime overhead only 8–17% over Multi-Hop vs. up to 1329% for rule-based PS-Agent.

What we take

The MDP skeleton transfers almost verbatim (entities → entity sets, relations → join edges, self-loop → STOP). Spurious paths exist on tables too, so we report join-path accuracy separately from EM. SCE's multiplicative reward is *not* potential-based (it can change the optimal policy), so we keep only its motivation; SCE's optimiser is the REINFORCE rung of our algorithm ladder.

B.4 StepSearch — dense step-wise rewards for PPO

Zheng, An, Wang, Wang, Wu. "StepSearch: Igniting LLMs' Search Ability via Step-Wise Proximal Policy Optimization." EMNLP 2025.

An LLM agent alternates think→search→answer over Wikipedia; trained on 19k MuSiQue-derived examples (Qwen2.5 3B/7B). *Step reward:* a memory vector keeps, for each gold document, the best TF-IDF cosine match achieved so far; the information gain is the non-negative increase $\Delta_t^i = \max(c_t^i - m_{t-1}^i, 0)$, averaged over gold documents; a redundancy penalty fires when a round's retrievals overlap the cumulative history. *Global reward:* answer F1 plus overlap of queries with gold keywords. Gains of 11.2 and 4.2 absolute points (3B, 7B) over global-reward RL.

Observation. Because m_t^i is a running maximum, $\sum_t \Delta_t^i = m_T^i - m_0^i$: information gain is shaping with potential $\Phi = \frac{1}{n}\sum_i m_t^i$ and $\gamma = 1$, but $\Phi(s_T) \neq 0$, so it implicitly adds a terminal coverage bonus to the objective.

What we take

On tables the same signal is *exact* (set overlap with executed gold prefixes) and free (no synthetic pipeline). We choose deliberately between invariant shaping ($\Phi(\text{terminal}) = 0$, our default) and an explicit coverage objective. Redundancy penalties become unnecessary because re-joining a visited table is masked.

B.5 GRPO on tables — why critic-free RL stalls

Kang, Wu, Wang, et al. "Can GRPO Boost Complex Multimodal Table Understanding?" EMNLP 2025.

Diagnoses GRPO's struggles on tables as low initial policy accuracy plus coarse rewards; proposes a three-stage framework: warm-up, perception-alignment GRPO with a continuous tree-edit-distance (TEDS) reward, and hint-completion GRPO that rewards only the residual steps after a hint.

Mechanism. GRPO's advantage is $\hat{A}_i = (r_i - \text{mean}(r_{1:G}))/(\text{std}(r_{1:G}) + \epsilon)$; if all G rollouts of a question receive the same reward, every $\hat{A}_i = 0$ and the question contributes no gradient. With a sparse 0/1 reward and a random initial policy, most groups are all-zero.

What we take

Justifies BC warm-start for every arm; hint completion maps to a *gold-prefix curriculum* (start rollouts after k gold hops, anneal $k \to 0$); and gives a measurable hypothesis—log the fraction of zero-advantage groups for GRPO.

B.6 Rewarding Progress — the theory behind dense rewards

Setlur, Nagpal, Fisch, Geng, Eisenstein, Agarwal, Agarwal, Berant, Kumar. "Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning." ICLR 2025.

A step's process reward should measure *progress*: the change in the probability of eventually succeeding, measured under a *prover* policy μ distinct from the base policy. Even weaker provers help if they distinguish steps and are not too misaligned. Process advantage verifiers were >8% more accurate and 1.5–5× more compute-efficient than outcome reward models in search, and gave ∼6× sample efficiency in online RL.

Connection. Progress is $A_\mu(s,a) = Q_\mu(s,a) - V_\mu(s)$. With deterministic transitions and no intermediate environment reward, $Q_\mu(s_t,a_t) = V_\mu(s_{t+1})$, hence

$$A_\mu(s_t,a_t) = V_\mu(s_{t+1}) - V_\mu(s_t),$$

exactly potential-based shaping with $\Phi = V_\mu$ and $\gamma = 1$.

What we take

Our structural potential is (a graded version of) the value of an *oracle prover* that completes the gold path; since wrong anchors/joins are irreversible, the oracle's value drops to 0 after a mistake, so it does distinguish steps. A learned alternative $\Phi = V_{\mathrm{BC}}$ needs only gold answers, not gold SQL (stretch goal).

B.7 VinePPO — is the critic accurate?

Kazemnejad, Aghajohari, Portelance, Sordoni, et al. "VinePPO: Refining Credit Assignment in RL Training of LLMs." ICML 2025.

Measures PPO's value network directly on LLM reasoning and finds it gives poor return estimates and barely beats random when ranking alternative next steps. Replaces it with unbiased Monte-Carlo estimates by resetting to an intermediate prefix and sampling K continuations, $\hat{V}(s_t) = \frac{1}{K}\sum_k R(\tau_t^{(k)})$; everything else is standard PPO. Outperforms PPO and baselines on MATH/GSM8K with up to 3× less wall-clock.

What we take

Sharpens our research question into "*when* does a learned critic help?" VinePPO's negative result comes from long token-level horizons; our regime is the opposite ($H \le 5$, structured states, ≤25 actions). We adopt (i) its diagnostic protocol (critic error and step-ranking accuracy against MC values) and (ii) an MC-value PPO arm. Our environment is cheaply resettable: SQLite transitions are deterministic, and reader outputs are cached by sub-table hash.

B.8 JAR — inferring join edges

Chen, Zhang, Roth. "Is Table Retrieval a Solved Problem? Exploring Join-Aware Multi-Table Retrieval." ACL 2024.

Argues many questions need a join plan that cannot be read off the question, and that FK constraints are often not given. Decomposes the question into (concept, attribute) pairs (GPT-3.5, bi-encoder scoring against columns) and scores column joinability as

$$\omega_{ij}^{kl} = \big(e(c_k,c_l) + j(c_k,c_l)\big)\cdot \max\{u(c_k),u(c_l)\},$$

with e schema similarity, j value Jaccard and u uniqueness (distinct/rows; a PK has $u = 1$). A mixed-integer program selects K connected tables (max-flow constraint) maximising relevance, coverage and compatibility. Spider top-2 retrieval F1: 78.9 (DTR) → 84.5 (inferred joins) → 89.6 (gold FKs). Gains are largest with ≥ 3 tables and bridging tables. Gold SQL that joins two non-unique columns (creating false pairings) was removed.

What we take

ω gives *inferred* candidate join edges (gold FKs used only in the reward) and per-edge features (e, j, u). JAR with a free MIP solver is our static, jointly-optimised structural baseline. We copy its data-cleaning rule: assert $\max(u_k, u_l) = 1$ on every gold join.

B.9 TableRAG — grounding anchors in large tables

Chen, Miculicich, Eisenschlos, et al. "TableRAG: Million-Token Table Understanding with Language Models." NeurIPS 2024.

Retrieves at two fine granularities instead of encoding whole rows/columns: *schema retrieval* (column name, type, min/max or top-3 categories) and *cell retrieval* over a database of *distinct* (column, value) pairs, capped to the B_{cell} most frequent (default 10,000), with LM query expansion and top-$K = 5$ per query. With GPT-3.5 it reaches 49.2 on ArcadeQA (tables averaging ∼79k rows) vs. 4.6 for reading the whole table; larger K did not reliably help.

What we take

The anchor action set is TableRAG's cell index restricted to the frontier tables (excluding ID columns with $u \approx 1$, which are reached by joins). Column summaries become state features. Random-rows and schema-only baselines at equal budget. Its program-aided solver supports "execute the aggregate" as a legitimate design choice.

B.10 TabSQLify — the one-shot select-then-read competitor

Nahid, Rafiei. "TabSQLify: Enhancing Reasoning Capabilities of LLMs Through Table Decomposition." NAACL 2024.

An LLM writes SQL to extract a small sub-table, then answers from it (64.7% WikiTQ, 79.5% TabFact with GPT-3.5); numbers and dates are normalised first. Architecturally it is our pipeline with the learned selector replaced by one-shot text-to-SQL.

What we take

The SQL-extract-then-read baseline (same reader, same budget), and value normalisation for EM.

B.11 Synthesis and research gap

MMQA shows the failure is concentrated at join and key decisions; Chain-of-Table and SCE show reasoning over structure is naturally sequential, but the former learns nothing and the latter lives on knowledge graphs with non-invariant rewards. StepSearch shows dense step rewards make PPO work for multi-hop retrieval, at the cost of a synthetic supervision pipeline; Kang et al. show critic-free GRPO stalls on tables when rewards are sparse; VinePPO questions whether learned critics are accurate at all; Setlur et al. show dense rewards should measure progress, which is potential-based shaping.

The gap we fill

No work learns a policy for anchor and join-key traversal in multi-table QA, or exploits that relational databases make exact, policy-invariant process rewards free (by executing gold-SQL prefixes). And nobody has tested whether a learned critic pays off in the short-horizon, small-action regime that such schema-constrained MDPs create—the regime opposite to where VinePPO found critics to fail.

Table 1: Reward taxonomy: which dense rewards preserve the optimal policy.

Reward term	Source	Potential-based?	Consequence
Information gain	StepSearch	almost ($\Phi(s_T) \neq 0$)	adds a terminal coverage bonus
Redundancy penalty	StepSearch	no	objective change (intended)
Consistency × accuracy	SCE	no	can change the optimum
Progress A_μ	Setlur et al.	**yes** ($\Phi = V_\mu$)	policy-invariant guidance
Gold-structure Jaccard, $\Phi(\text{term}) = 0$	**ours**	**yes**	invariant; progress under an oracle prover

C MDP Formulation (Final)

C.1 Design decisions

Part A left six design decisions open. Table 2 fixes them for the main experiments; alternatives are kept as ablations or stretch goals.

Table 2: Resolved design decisions.

Decision	Main setting	Rationale / alternative
Candidate join edges	**Inferred** via JAR joinability ω; gold FKs used only in the reward	keeps structure a real decision; gold-FK environment reported as an ablation
Environment instance	gold tables **+** k **distractor tables** from the same Spider database ($k \leq 3$)	with 2–3 given tables the join choice is near-trivial
Selection granularity	rows via anchors/joins; **columns by a fixed rule**	learned projection is a stretch goal
Non-monotone questions	kept; **reported separately**	they stress a different capability
Aggregation	done by the **reader**	execute-the-aggregate variant as ablation
Budget	hard $B = 4{,}096$ tokens	sweep $B \in \{1\text{k}, 2\text{k}, 4\text{k}, 8\text{k}\}$ if time permits

C.2 Episode instance

An episode is defined by $(Q, D_{\text{inst}}, q^\star, a^\star)$ where $D_{\text{inst}} = D_{\text{gold}} \cup D_{\text{distr}}$. Each MMQA instance is mapped to its Spider database by schema matching; distractors are sampled from the

remaining tables of that database (fixed seed at evaluation, resampled during training as augmentation). Instances are split **by database** (train/dev/test $\approx$ 60/10/30, stratified by 2-/3-table), so no schema is seen at both training and test time. Before any episode is run, the environment builds three training-free indexes (Part E): the candidate-edge set $\mathcal{E}$ (JAR ω above a threshold τ_ω chosen for $\geq$98% gold-edge recall on train), the anchor cell index (TableRAG-style distinct values, ID-like columns excluded), and cached frozen embeddings.

C.3 Formal definition

The MDP is $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, H, \gamma)$ with $H = 5$ and $\gamma = 1$, deterministic P.

State. At step t,

$$s_t = \big(Q,\ \mathcal{V}_t,\ \mathcal{J}_t,\ \mathcal{P}_t,\ t\big),$$

where $\mathcal{V}_t$ is the set of visited tables (the join tree), $\mathcal{J}_t \subseteq \mathcal{E}$ the joins taken, and $\mathcal{P}_t$ the anchor predicates applied. These determine the current intermediate relation

$$R_t = \sigma_{\mathcal{P}_t}\Big(\bowtie_{e \in \mathcal{J}_t} \mathcal{V}_t\Big),$$

so the state is Markov. The *observation* fed to the network is: the question embedding; one token per visited table (name and column summary embeddings); one token per applied predicate; and scalars t/H, $\log(1 + |R_t|)$, $\mathrm{tok}(R_t)/B$, $|\mathcal{V}_t|$, $|\mathcal{P}_t|$. The gold query is never observed.

Actions. Four factorised action types (Chain-of-Table style), each instantiated from a small candidate set:

Type	Candidates	Typical count
Seed(T)	start from all rows of table T (only at $t = 0$; for questions with no filter)	$m + k \leq 6$
Anchor$(T.c$ op $v)$	categorical: top-8 (column, value) pairs from the cell index by similarity to Q; numeric: numbers parsed from Q $\times$ top-2 numeric columns $\times$ $\{=, <, >\}$; restricted to visited tables for $t \geq 1$	≤ 14
Join(e)	edges $e \in \mathcal{E}$ connecting a visited table to an unvisited one	≤ 6
Stop	materialise and hand to the reader	1

An Anchor at $t = 0$ implicitly seeds its table. $|\mathcal{A}(s_t)| \leq 25$ throughout.

Action masking. Invalid actions are masked, not penalised: $t = 0$ allows only Seed/Anchor; Join only to unvisited tables (so no re-joins and no redundancy penalty is needed); at most two anchors and no duplicate predicate; Stop only for $t \geq 1$; at $t = H - 1$ only Stop.

Action features. Each candidate carries its type, a text embedding (table; "$T.c$ op v"; or the two column embeddings of an edge), similarity to Q, JAR's ω, e, j, u for edges, and *look-ahead cardinalities* $\log(1 + |R_{t+1}|)$ and $\mathrm{tok}(R_{t+1})/B$ obtained by a cheap SQL `COUNT`. The look-ahead uses no gold information (it is cardinality estimation, as in a query optimiser) and is what lets the policy be selectivity-aware.

Transitions. Deterministic and executed in SQLite: Seed(T): $R_{t+1} = T$; Anchor(p): $R_{t+1} = \sigma_p(R_t)$; Join(e): $R_{t+1} = R_t \bowtie_e T'$. Implemented as composed SQL views; an empty R_{t+1} is allowed (it will earn no reward).

Termination and materialisation. STOP (or reaching H) triggers materialisation: $S = \pi_C(R_T)$, where C = columns on the path (join keys, anchor columns) $\cup$ the top-4 columns of visited tables by schema similarity to Q. S is serialised as a Markdown table and truncated *hard* to B tokens in a deterministic row order (overflow is flagged). The frozen reader receives (Q, S).

C.4 Reward

Outcome. $R_{\text{ans}} = \text{EM}_{\text{norm}}(\mathcal{R}(Q,S), a^\star) \in \{0,1\}$, with TabSQLify-style normalisation of numbers and dates and set-equality for List answers (a set-F1 variant is an ablation).

Structural potential. Let $G^\star$ be the gold structural items obtained from the parsed gold SQL: gold join edges (unordered column pairs) and gold anchor predicates (column, operator, normalised value). Let $I(s)$ be the items taken by the policy in state s. Define

$$\Phi(s) = \frac{|I(s) \cap G^\star|}{|I(s) \cup G^\star|} \text{ (Jaccard)}, \qquad \Phi(s_0) = 0, \qquad \Phi(s_{\text{term}}) \equiv 0.$$

Jaccard is order-invariant (valid alternative join orders are not punished) and decreases when a wrong item is added.

Per-step reward.

$$r_t = \beta\left[\Phi(s_{t+1}) - \Phi(s_t)\right] + \mathbb{1}[a_t = \text{STOP}]\left(R_{\text{ans}} - \lambda_c \frac{\text{tok}(S)}{B}\right), \tag{4}$$

with $\beta = 0.5$ and $\lambda_c = 0.1$ by default. The cost term deliberately changes the objective (it trades accuracy for tokens); the shaping term does not.

Proposition (policy invariance of the shaping term)

In this finite-horizon episodic MDP with $\gamma = 1$ and $\Phi(s_{\text{term}}) = 0$, the shaping terms of any trajectory sum to $\beta[\Phi(s_{\text{term}}) - \Phi(s_0)] = 0$. Hence for every policy π, $J_{\text{shaped}}(\pi) = J(\pi)$ and the optimal policies coincide. Shaping only redistributes credit over time, reducing variance for per-state baselines.

Proof sketch. Telescoping: $\sum_{t=0}^{T-1}[\Phi(s_{t+1}) - \Phi(s_t)] = \Phi(s_T) - \Phi(s_0) = 0 - 0$. The identity holds pathwise, so it holds in expectation under any π. □

Two remarks. (i) The last shaping term before termination is $-\beta\Phi(s_{T-1})$: stopping "returns" the potential, and the outcome reward at STOP is what makes stopping worthwhile. (ii) Under the oracle-prover reading (Part B.6), Φ is a graded proxy for $V_{\mu^\star}(s)$. Because anchors and joins are irreversible, a wrong step lowers Φ and it can never be recovered, so the potential discriminates steps (we verify empirically the fraction of transitions with $\Delta\Phi \neq 0$).

Reward variants. **R0** sparse ($\beta = 0$); **R1** structural shaping (default); **R2** (stretch) prover potential $\Phi = \hat{V}_{\text{BC}}$ from Monte-Carlo rollouts of the BC policy, which needs gold answers only.

C.5 Oracle construction and data integrity

The gold SQL is parsed with `sqlglot` into tables, join conditions (ON clauses and column-equality WHERE terms), anchor predicates (column op literal) and a monotonicity flag (subqueries, NOT IN, EXCEPT, ORDER BY … LIMIT, aggregates). Each prefix of the canonical plan is executed to obtain $E_t^\star$. Four checks are *asserted*, and failures are dropped and counted, never silently repaired:

1. executing $q^\star$ reproduces the annotated $a^\star$ (after normalisation);
2. every gold join satisfies $\max(u_k, u_l) = 1$ (JAR's rule against false pairings);
3. every gold edge is in the candidate set $\mathcal{E}$ (*edge-candidate recall*);
4. every gold anchor is among the anchor candidates (*anchor-candidate recall*).

Items 3–4 are not filters but environment diagnostics: their conjunction gives the *reachability* rate, the fraction of instances whose gold trajectory lies inside the action space, which upper-bounds achievable join-path accuracy.

BC oracle trajectories. A canonical order is used: anchors first (seeding their table), then joins in breadth-first order from the anchored table, then STOP; questions without an anchor start with SEED on the first FROM table. BC imitates this one order; RL is free to find others.

C.6 Worked episode

Figure 1 traces the Part A example through the MDP, including potentials and rewards.

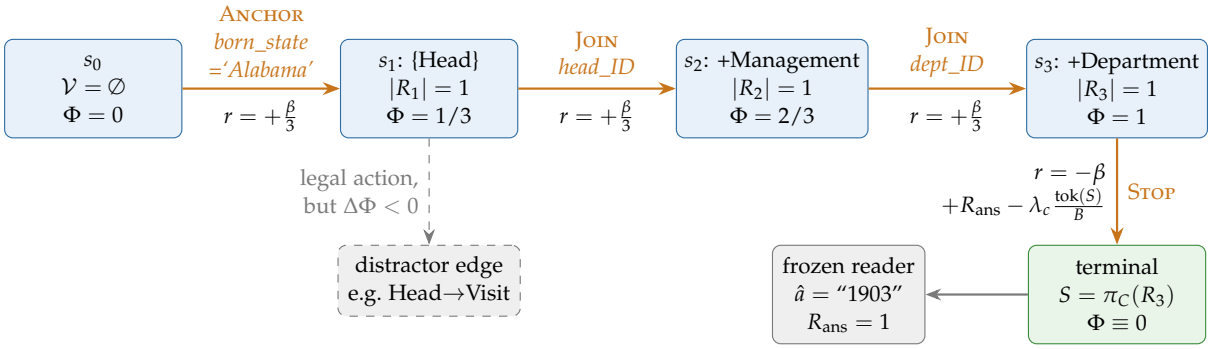


Figure 1: The worked example as an MDP episode. Shaping contributes $+\beta/3$ per correct step and $-\beta$ at STOP, summing to zero; the return is $R_{\text{ans}} - \lambda_c \text{ tok}(S)/B$. A distractor join is a legal action whose Jaccard potential decreases.

C.7 Properties of the MDP

- **Short horizon**: 2–4 decisions for MMQA questions (anchor, one or two joins, stop), capped at $H = 5$.
- **Small, variable action sets** ($\leq$25), handled by a pointer-style scorer with masking.
- **Deterministic, cheaply resettable** transitions (SQL views): Monte-Carlo value estimation from any state costs only reader calls, which are cached.
- **Irreversibility**: wrong anchors/joins cannot be undone, which makes the oracle potential discriminative and makes credit assignment to early steps the crux.
- **Two-level cost**: environment steps are nearly free; the reader call at STOP dominates compute and is the unit for matched-budget comparisons.

D Evaluation Metric Suite

Metrics are chosen so that each one either measures the objective (2) directly or isolates one factor of the decomposition (3). Deterministic metrics are headline; LLM-judged metrics are

secondary and audited.

Table 3: Final metric suite.

Metric	Role	Why (or what changed)
Normalised EM (case/whitespace/number/date normalisation; set-equality for List)	headline	deterministic and reproducible; MMQA's primary metric; StepSearch likewise avoids LLM judges
Set-F1 over answer items	headline	replaces token-F1: deterministic partial credit for List/Count/Numerical answers
Join-path exact accuracy	headline (process)	measures the policy's own output; exposes spurious paths that EM hides
Sub-table answer coverage (does S contain the gold answer lineage?)	headline (decoupling)	isolates selector quality from reader quality at no LLM cost
Join-edge precision / recall	diagnostic	replaces "FK/PK accuracy"; an edge is an (FK, PK) column pair so PK correctness is implied; not numerically comparable to MMQA's question-free FKS
Per-hop entity recall / precision vs. $E_t^\star$	diagnostic	localises failures to anchor vs. hop 1 vs. hop 2; free from the oracle
Reader input tokens per question	efficiency	central to the "selection is cheaper" thesis (#calls is always 1; latency is hardware-dependent)
Partial Match (LLM judge)	secondary, audited	pinned judge and prompt; 100-sample hand audit, since judge–human agreement falls for weaker readers
RL diagnostics: entropy, KL to previous policy and to BC, value explained variance, return curves, zero-advantage fraction (GRPO)	training only	explain *why* an arm wins; not comparison metrics

Dropped. Standalone PK accuracy (not a decision our policy makes), Text-to-SQL ROUGE/BLEU (not our task; MMQA shows surface metrics mislead), #LLM calls (constant) and latency (hardware-dependent).

Environment diagnostics (reported once per split, Part C.5): edge-candidate recall, anchor-candidate recall, and reachability. **Critic diagnostics** (Part E): value MAE and step-ranking accuracy against Monte-Carlo estimates.

Reporting slices. All headline metrics are reported overall and by 2-/3-table subset, question type (Select, List, Count, Numerical), and monotone vs. non-monotone.

Statistical protocol. Three seeds per RL arm; paired bootstrap over test questions (10,000 resamples) for EM differences, with 95% intervals. Any PPO-vs-alternative gap inside seed variance is reported as "no significant difference".

E System and RL Architecture

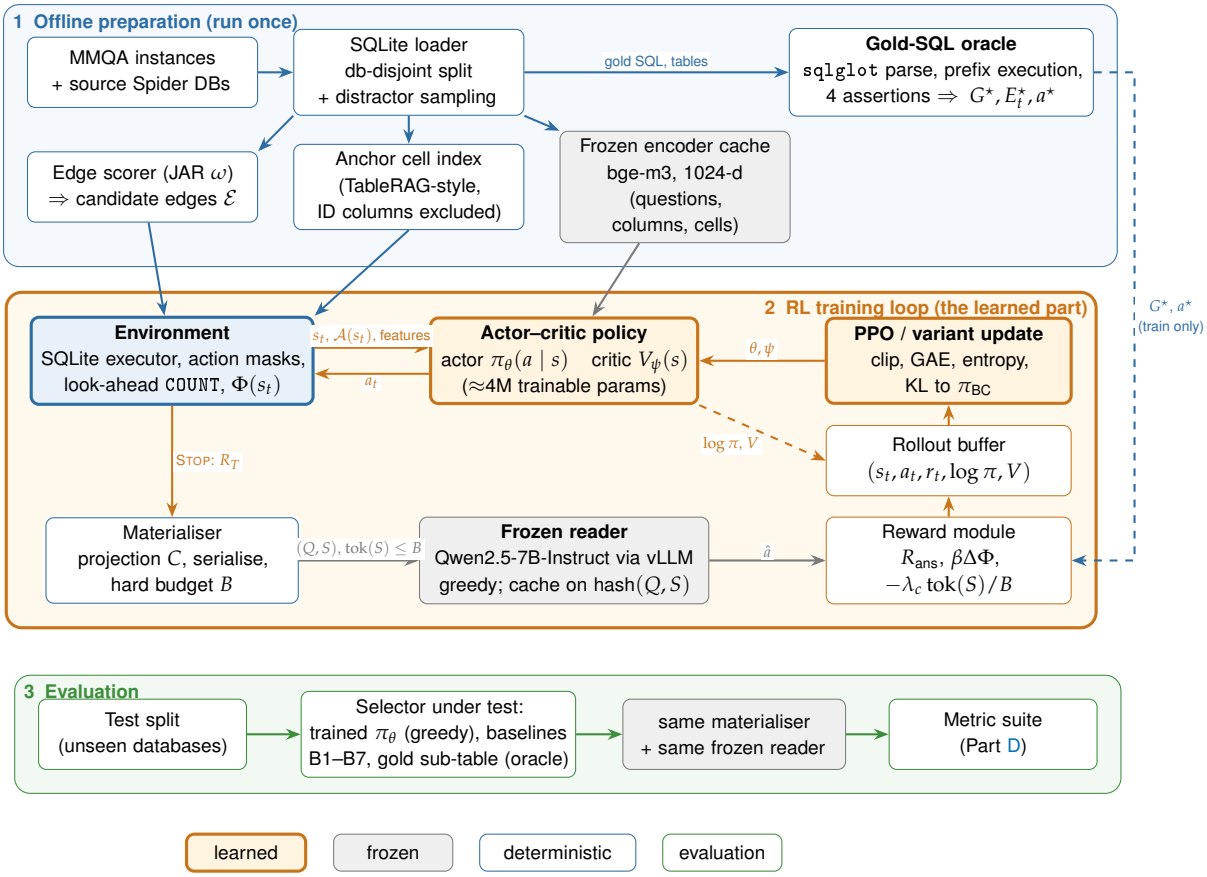


Figure 2: End-to-end pipeline. (1) Offline: data loading, db-disjoint split, gold-SQL oracle, and the training-free indexes that define the action space. (2) RL loop: the policy interacts with a deterministic SQLite environment; at STOP the sub-table is materialised under budget B and read by the frozen reader; rewards combine the answer, potential-based shaping from the oracle, and a token cost. (3) Evaluation: every selector (learned, baseline or oracle) goes through the same materialiser and reader.

E.1 Overall pipeline

Figure 2 shows where every component sits. Only the orange band is learned; everything else is either built once offline (blue) or frozen (grey). The same materialiser and reader are shared by the RL policy, every baseline and the oracle upper bound, so all comparisons differ *only* in how the sub-table is selected.

E.2 Policy network

The policy is deliberately small: all language understanding is delegated to a frozen encoder whose embeddings are pre-computed once, so the trainable part is a few million parameters and a PPO update takes milliseconds. Figure 3 shows the architecture.

Specification. State tokens: one for Q, one per visited table, one per predicate, one scalar token (MLP of the scalar features), plus [CLS]; at most ∼8 tokens. Action scoring is permutation-invariant over candidates, so variable-size action sets need no padding tricks beyond masking. Trainable parameters: ≈2M (actor) + ≈2M (critic). This is smaller than the 10–50M we estimated earlier, because embeddings are cached rather than fine-tuned.

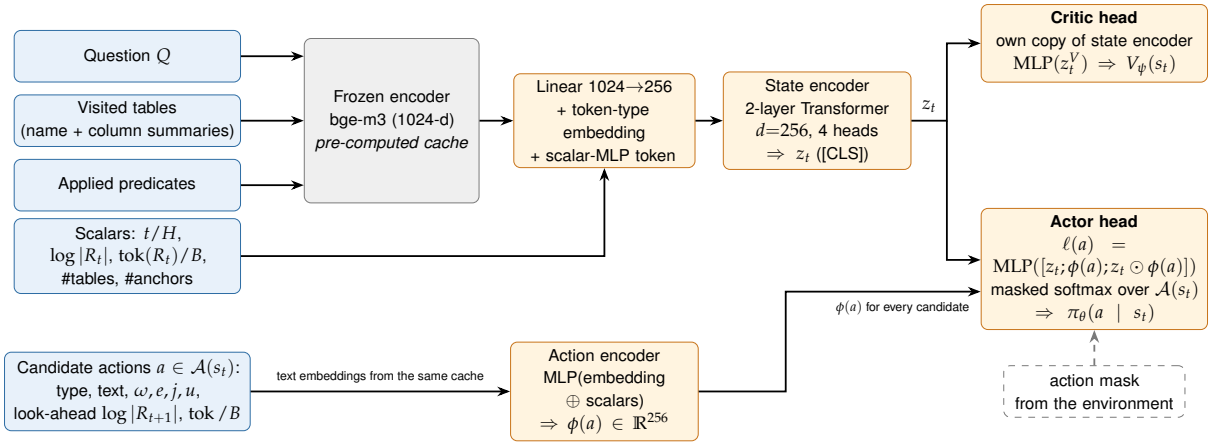


Figure 3: Actor–critic network. The actor is a pointer-style scorer over a variable candidate set, masked by the environment; the critic has its own copy of the state encoder, so the two losses do not interfere through shared weights.

E.3 The PPO actor–critic skeleton

Objective. With probability ratio $\rho_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ and advantage $\hat{A}_t$,

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{5}$$

$$L^V(\psi) = \mathbb{E}_t\Big[\max\big((V_\psi(s_t) - \hat{G}_t)^2,\ (\text{clip}(V_\psi(s_t), V_{\text{old}} \pm \epsilon_V) - \hat{G}_t)^2\big)\Big], \tag{6}$$

$$L(\theta, \psi) = -L^{\text{clip}} + c_V L^V - c_H\, \mathcal{H}[\pi_\theta] + c_{\text{KL}}\, \text{KL}\big(\pi_\theta \,\|\, \pi_{\text{BC}}\big). \tag{7}$$

Advantages (GAE). $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$ with $V_\psi(s_{\text{term}}) = 0$, $\hat{A}_t = \sum_{l \geq 0}(\gamma\lambda)^l \delta_{t+l}$, $\hat{G}_t = \hat{A}_t + V_\psi(s_t)$, $\gamma = 1$, $\lambda = 0.95$, normalised per batch.

Shaping is a hand-built baseline

With $\gamma = 1$, the shaped reward-to-go satisfies $G'_t = \sum_{t' \geq t} r'_{t'} = G_t - \beta\Phi(s_t)$. So REINFORCE on shaped rewards is *exactly* REINFORCE on unshaped rewards with state-dependent baseline $\beta\Phi(s_t)$, and PPO's critic only has to learn the residual $V - \beta\Phi$. Two predictions follow. (i) Shaping helps only methods that use per-step returns or values. (ii) **GRPO is provably unaffected by our shaping**: it uses the trajectory return, whose shaping component is zero pathwise. R0 vs. R1 therefore also tests where the benefit of dense rewards actually enters.

Table 4: Default hyperparameters (tuned only on dev).

Parameter	Value	Parameter	Value
Horizon H / discount γ	5 / 1.0	GAE λ	0.95
Questions per iteration M	128	Iterations N_{iter}	300 (≈38k episodes)
PPO epochs K_{ep} / minibatch	4 / 256 transitions	Clip ϵ / value clip ϵ_V	0.2 / 0.2
LR actor / critic (Adam)	3×10^{-4} / 1×10^{-3}	Grad-norm clip	0.5
Entropy c_H	0.01 → 0.001 (linear)	Value coef. c_V	0.5
KL to BC c_{KL}	0.02	Shaping β / cost λ_c	0.5 / 0.1
Budget B	4,096 tokens	Curriculum p_k	0.5 → 0 over 100 iters
Reader	Qwen2.5-7B-Instruct, greedy	Max new tokens	64

Algorithm 1 PPO actor–critic for entity selection (main arm)

Init $\theta \leftarrow \theta_{\text{BC}}$ (behaviour cloning on oracle trajectories); ψ random; reader cache $\mathcal{C} \leftarrow \emptyset$
for iteration $= 1, \ldots, N_{\text{iter}}$ **do**
  Sample M training questions; with prob. p_k start at the state after k gold steps (curriculum, $p_k \to 0$)
  for each question **do**
    repeat observe s_t, $\mathcal{A}(s_t)$, mask; sample $a_t \sim \pi_\theta$; step SQLite; $r_t \leftarrow \beta\Delta\Phi$ **until** STOP or $t = H$
    $S \leftarrow$ materialise(R_T); $\hat{a} \leftarrow \mathcal{C}[\text{hash}(Q, S)]$ or call reader (vLLM) and cache
    add $R_{\text{ans}} - \lambda_c \, \text{tok}(S)/B$ to the terminal reward; store $(s_t, a_t, r_t, \log \pi_\theta(a_t|s_t), V_\psi(s_t))$
  end for
  Compute $\hat{A}_t, \hat{G}_t$ by GAE; normalise $\hat{A}$
  for epoch $= 1, \ldots, K_{\text{ep}}$ **do**
    for minibatch **do**
      gradient step on $L(\theta, \psi)$ (Adam, grad-norm clip)
    end for
  end for
  Log entropy, KL, value explained variance, returns; every 25 iters evaluate on dev
end for

E.4 PPO variants: a ladder of baselines

All variants share the network, BC initialisation, environment, reward and reader budget; they differ only in how the advantage is estimated and how the update is constrained. Figure 4 orders them by the granularity of the baseline, which is the variable our research question is about.

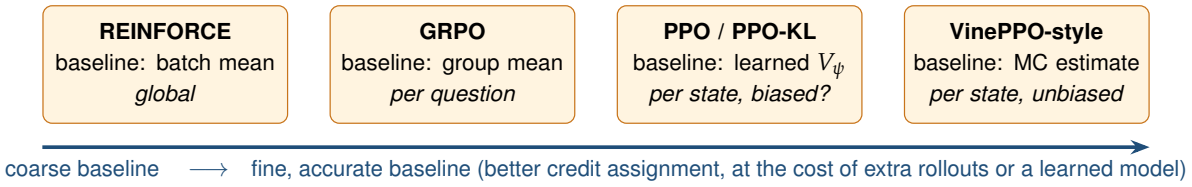


Figure 4: The baseline-granularity ladder. PPO vs. VinePPO isolates the *accuracy* of the per-state baseline; GRPO vs. PPO isolates its *granularity*; REINFORCE is the floor (and SCE's optimiser).

Where each variant plugs in. Figure 5 shows the shared training loop; variants replace only the advantage estimator (step 3) and the update rule (step 4). V4 additionally uses the environment's reset capability during rollouts; its extra reader calls count against the matched budget.

Further ablations (main arm only). BC initialisation vs. from scratch; curriculum on/off; shared vs. separate critic trunk; inferred vs. gold-FK candidate edges; reward R2 (prover potential, stretch).

Critic diagnostics. Every 25 iterations, sample 500 visited states, compute $\hat{V}_{\text{MC}}$ with $K = 8$ resets, and report (i) $|V_\psi - \hat{V}_{\text{MC}}|$ and (ii) pairwise step-ranking accuracy: at a state with two or more candidate actions, whether V_ψ orders the successor states as MC does. This is VinePPO's protocol, applied in the opposite regime.

Table 5: The five algorithm arms.

Arm	Advantage $\hat{A}_t$	Update constraint	What it tests
V1 PPO-Clip (main)	GAE with learned critic V_ψ	ratio clipping, $\epsilon = 0.2$	reference actor–critic
V2 PPO-KL	same as V1	adaptive KL penalty $\beta_k \, \mathrm{KL}(\pi_{\text{old}} \| \pi_\theta)$, target 0.01	does the trust-region *form* matter at this scale?
V3 GRPO	G=8 rollouts per question; $(R_i - \bar{R})/(\sigma_R + \epsilon)$ applied to every step; no critic	ratio clipping + KL to π_{BC}	per-question baseline; zero-advantage fraction
V4 VinePPO-style	$\hat{V}(s_t) = \frac{1}{K}\sum_{k=1}^{K} G(\tau_t^{(k)})$ from K=4 resets; $\hat{A}_t = r_t + \hat{V}(s_{t+1}) - \hat{V}(s_t)$; no critic	ratio clipping	accurate per-state values without a learned model
V5 REINFORCE	reward-to-go minus batch-mean baseline	none (1 gradient step per batch)	floor; equals SCE's optimiser

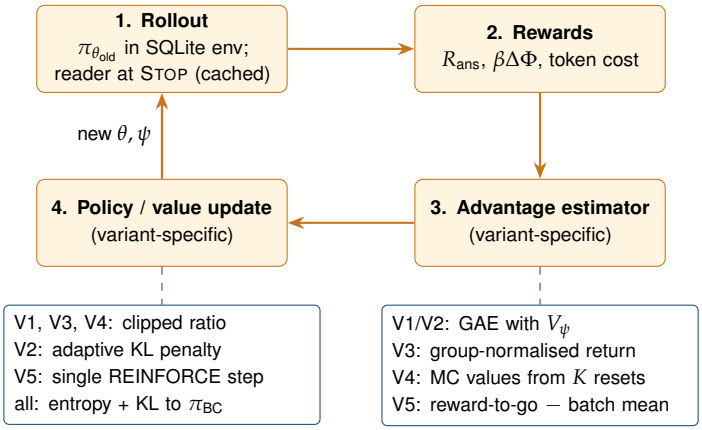


Figure 5: The shared training loop with variant plug-in points.

E.5 Baselines and experimental protocol

Baseline	Description (same materialiser, reader and budget B)
B1 Truncated table	all tables serialised, cut to B tokens
B2 Random rows	uniformly sampled rows per table up to B
B3 Schema only	column names, types and summaries, no rows
B4 JAR-lite	static MIP selection of tables and edges (free CBC solver) + top-1 anchor from the cell index
B5 Prompted planner	the frozen reader chooses actions over our action API with in-context examples (Chain-of-Table style)
B6 SQL-extract-then-read	the reader writes extraction SQL; its result is read (TabSQLify style)
B7 Behaviour cloning	supervised policy on oracle trajectories (also the RL initialisation)
Oracle	gold sub-table $\Rightarrow$ the upper bound on accuracy for this reader

Matched budget. All RL arms receive the same number of reader calls (cache misses, including V4's Monte-Carlo rollouts); learning curves are plotted against reader calls, not

iterations. **Ablation grid:** 5 arms $\times$ {R0, R1} $\times$ 3 seeds $=$ 30 runs.

E.6 Compute budget

Item (A100-class GPU-hours)	Estimate
Embedding cache (questions, columns, distinct cells)	3
BC training + reader-harness validation	3
30 main RL runs ($\approx$23k uncached reader calls each at $\sim$10k calls/GPU-h, + updates)	90
Extra ablations ($\approx$12 runs)	36
Baselines B1–B7 + oracle on dev/test; PM judge	15
Periodic dev evaluation + critic diagnostics	10
Debugging and tuning	40
Subtotal / **with $\sim$25% contingency**	197 / $\approx$**250**

The reader is served with vLLM on one GPU (bf16 on 40–80 GB, or AWQ-int4 on a 24 GB card at roughly two-thirds the throughput); the 4M-parameter policy trains on the same GPU. Two GPUs let the two members run arms and seeds in parallel, so the budget fits in weeks 5–9 at about 25 GPU-hours per GPU per week.

References

[1] J. Wu, L. Yang, D. Li, Y. Ji, M. Okumura, Y. Zhang. MMQA: Evaluating LLMs with Multi-Table Multi-Hop Complex Questions. *ICLR 2025*. https://openreview.net/forum?id=GGlpykXDCa

[2] Z. Wang et al. Chain-of-Table: Evolving Tables in the Reasoning Chain for Table Understanding. *ICLR 2024*.

[3] Y. Huang, Y. Liu, Q. Liu, R. Hou, T. Dai. SCE: Semantic Consistency Enhanced Reinforcement Learning for Multi-Hop Knowledge Graph Reasoning. *Findings of EMNLP 2025*, pp. 5413–5425. https://aclanthology.org/2025.findings-emnlp.289/

[4] X. Zheng, K. An, Z. Wang, Y. Wang, Y. Wu. StepSearch: Igniting LLMs' Search Ability via Step-Wise Proximal Policy Optimization. *EMNLP 2025*. https://aclanthology.org/2025.emnlp-main.1106/

[5] X. Kang, S. Wu, Z. Wang, et al. Can GRPO Boost Complex Multimodal Table Understanding? *EMNLP 2025*. https://aclanthology.org/2025.emnlp-main.637/

[6] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, A. Kumar. Rewarding Progress: Scaling Automated Process Verifiers for LLM Reasoning. *ICLR 2025*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/98711dea460bdefe0e651ca23ec98ba2-Abstract-Conference.html

[7] A. Kazemnejad, M. Aghajohari, E. Portelance, A. Sordoni, et al. VinePPO: Refining Credit Assignment in RL Training of LLMs. *ICML 2025*, PMLR 267:29557–29590. https://proceedings.mlr.press/v267/kazemnejad25a.html

[8] P. B. Chen, Y. Zhang, D. Roth. Is Table Retrieval a Solved Problem? Exploring Join-Aware Multi-Table Retrieval. *ACL 2024*, pp. 2687–2699. https://aclanthology.org/2024.acl-long.148/

[9] S.-A. Chen, L. Miculicich, J. M. Eisenschlos, et al. TableRAG: Million-Token Table Understanding with Language Models. *NeurIPS 2024*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/88dd7aa6979e352fda7c4952ca8eac59-Abstract-Conference.html

[10] M. M. H. Nahid, D. Rafiei. TabSQLify: Enhancing Reasoning Capabilities of LLMs Through Table Decomposition. *NAACL 2024*, pp. 5725–5737. https://aclanthology.org/2024.naacl-long.320/

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[12] A. Y. Ng, D. Harada, S. Russell. Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping. *ICML 1999*.

[13] T. Yu et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP 2018*. https://aclanthology.org/D18-1425